```
template<
    typename Dispatcher,
    typename Msg,
    typename Func>
friend class TemplateDispatcher;

void wait_and_dispatch()
{
    for(;;)
    {
        auto msg=q->wait_and_pop();
        dispatch(msg);
    }
}

bool dispatch(
    std::shared_ptr<message_base> const& msg)
{
    if(dynamic_cast<wrapped_message<close_queue>*>(msg.get()))
    {
        throw close_queue();
    }
```

Allow TemplateDispatcher instances to access the internals

❶ **Loop, waiting for and dispatching messages**

❷ **dispatch() checks for a close_queue message, and throws**

The `dispatcher` instance that's returned from `wait()` will be destroyed immediately, because it's a temporary, and as mentioned, the destructor does the work. The destructor calls `wait_and_dispatch()`, which is a loop ❶ that waits for a message and passes it to `dispatch()`. `dispatch()` itself ❷ is rather simple; it checks whether the message is a `close_queue` message and throws an exception if it is; otherwise, it returns `false` to indicate that the message was unhandled. This `close_queue` exception is why the destructor is marked `noexcept(false)`; without this annotation the default exception specification for the destructor would be `noexcept(true)` ❹, indicating that no exceptions can be thrown, and the `close_queue` exception would thus terminate the program.

It's not often that you're going to call `wait()` on its own, though; most of the time you'll be wanting to handle a message. This is where the `handle()` member function ❸ comes in. It's a template, and the message type isn't deducible, so you must specify which message type to handle and pass in a function (or callable object) to handle it. `handle()` itself passes the queue, the current `dispatcher` object, and the handler function to a new instance of the `TemplateDispatcher` class template, to handle